

6장 임베딩

임베딩

- 텍스트 벡터화 (Text Vectorization) : 컴퓨터가 이해할 수 있도록 텍스트를 숫자로 변환하는 과정
 - 원-핫 인코딩 (One Hot Encoding)
 - 벡터의 희소성이 큼 → 컴퓨팅 비용 증가, 차원의 저주
 - 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않음
 - 빈도 벡터화 (Count Vectorization)
 - 워드 임베딩 (Word Embedding)
 - 동적 임베딩 (Dynamic Embedding)

언어 모델 (Language Model)

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델.

자기회기 언어 모델 (Autogressive Language Model)

주어진 문장 뒤에 나올 수 있는 문장은 매우 다양하기 때문에 완성된 문장 단위로 확률을 계산하는 것은 어려움. 이러한 문제를 해결하기 위해 문장 전체를 예측하는 방법 대신에 하나의 토큰 단위로 예측하는 모델

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측

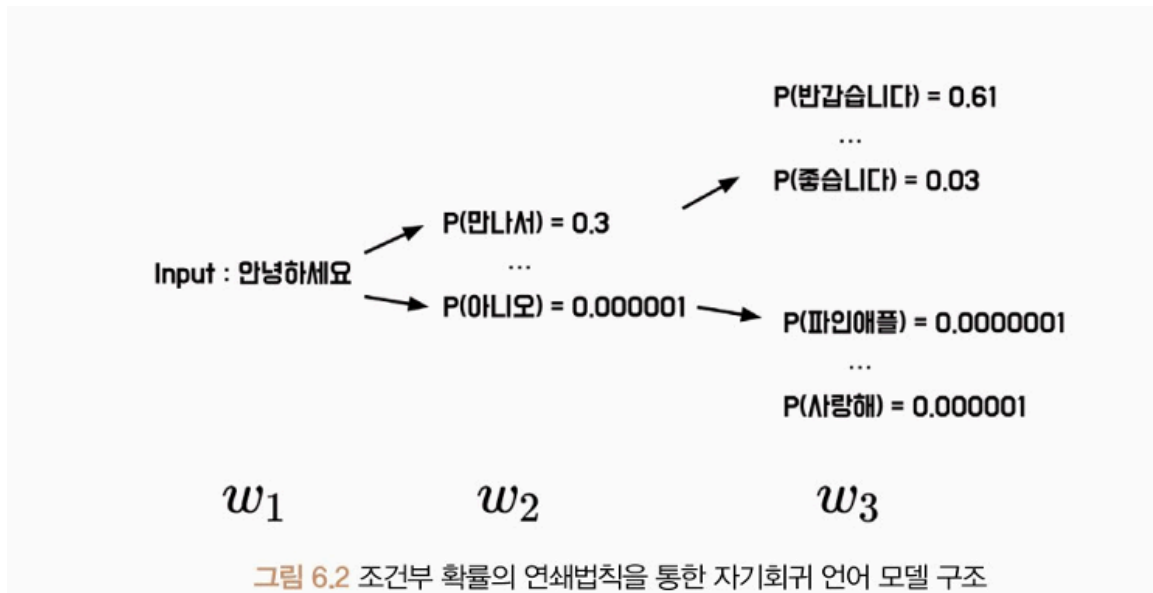
수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

- 예시



- 모델의 출력값이 다음 모델의 입력값으로 사용됨.

통계적 언어 모델 (Statistical Language Model)

언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측

- 마르코프 체인 (Markov Chain) : 빈도 기반의 조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간 전이 확률을 이용해 다음 상태를 예측



빈도 기반의 조건부 확률

주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률 계산

'안녕하세요' : 1000회 / '안녕하세요 만나서' : 700 회 / '안녕하세요 반갑습니다' : 100회

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

→ 단어의 순서와 빈도에만 기초해 문장의 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있음

→ 데이터 희소성 문제 (Data sparsity) : 관측한 적이 없는 데이터를 예측하지 못하는 문제

- 하지만 최근의 발전된 통계적 언어 모델은 기존에 학습한 데이터에서 패턴을 찾아 확률 분포를 생성하므로 이를 이용하여 새로운 문장을 생성할 수도 있다.

N-gram 모델

가장 기초적인 통계적 언어 모델. 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.

- N의 값에 따라 유니그램(Unigram), 바이그램(Bigram), 트라이그램(Trigram) 등으로 부름
- 예시

Unigram : <u>안녕하세요</u> <u>만나서</u> <u>진심으로</u> <u>반가워요</u>	안녕하세요. 만나서. 진심으로. 반가워요
Bigram : <u>안녕하세요 만나서</u> <u>진심으로 반가워요</u>	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : <u>안녕하세요 만나서 진심으로</u> <u>반가워요</u>	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

- N-gram 인 언어 모델은 모든 토큰을 사용하지 않고 N-1 개 토큰만을 고려해 확률을 계산한다.

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

w_t (=예측하려는 단어), $w_{t-1} \sim$ (=예측에 사용되는 이전 단어)

- N-gram 구현

```
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

```

[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',))]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]

```

• N-gram 사용예시



N-gram 사용예시

- 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석
- 관용적 표현
→ 자주 등장하는 연속된 단어나 구를 추출해 문맥을 파악
- 단어의 순서가 중요한 경우

TF-IDF(Term Frequency-Inverse Document Frequency)

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치.



BoW (Bag-of-Words)

문서나 문장을 단어의 집합으로 표현하는 방법. 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록한다. (원-핫 인코딩과는 달리 등장 빈도를 고려)

ex) ["That movie is famous movie", "I like that actor", "I don't like that actor"] →

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

하지만 모든 단어가 동일한 가중치를 가지므로 높은 성능을 기대하기는 어렵다.

- 문서 빈도와 역문서 빈도를 곱한 값 사용
문서 내에 단어가 자주 등장하고 전체 문서 내에 해당 단어가 적게 등장할수록 TF-IDF 값이 커짐

수식 6.8 TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

단어 빈도 (Term Frequency, TF)

문서 내에서 특정 단어의 빈도수를 나타내는 값

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

- 단순히 단어의 등장 빈도 수를 계산하기 때문에 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아진다.

문서 빈도 (Document Frequency, DF)

한 단어가 얼마나 많은 문서에서 나타나는지

수식 6.6 문서 빈도

$$DF(t, D) = count(t \in d : d \in D)$$

- DF 값이 높으면 특정 단어가 많은 문서에서 등장하고, 일반적으로 널리 사용되는 단어로 중요도가 낮을 수 있다.

역문서 빈도 (Inverse Document Frequency, IDF)

전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값.

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- 문서 빈도가 높을수록 해당 단어는 일반적이고 중요하지 않음 $x \rightarrow$ 역수를 취해 단어의 빈도 수가 적을수록 IDF 값이 커지게 보정
- 더 정교한 가중치를 얻기 위해 로그를 취한다.

TF-IDF 실습

- 사이킷런 라이브러리 사용
말뭉치를 쉽게 TF-IDF 형태로 변환할 수 있게 클래스를 제공

TF-IDF 클래스

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(  
    input="content",  
    encoding="utf-8",  
    lowercase=True,  
    stop_words=None,  
    ngram_range=(1, 1),  
    max_df=1.0,  
    min_df=1,  
    vocabulary=None,  
    smooth_idf=True,  
)
```

- input : 입력된 데이터의 형태
'context' (기본값) , 문자열 데이터 또는 바이트

'file' , 파일 객체
'filename' 파일 이름

- 인코딩(encoding)
 - 소문자 변환 (lowercase)
 - 불용어 (stop_words)
 - N-gram 범위 (ngram_range) : 사용할 n-gram 의 범위. (최솟값, 최댓값) 형태로 입력
 - 최댓값 문서 빈도 (max_df) : 전체 문서 중 일정 횟수 이상 등장하는 단어를 불용어 처리
 - 최솟값 문서 빈도 (min_df) : 전체 문서 중 일정 횟수 미만 등장하는 단어를 불용어 처리
 - 단어 사전 (Vocabulary) : 미리 구축한 단어사전 또는 새 단어사전
 - IDF 분모처리 (smooth_idf) : IDF 계산 시 분모에 1을 더함
- TF-IDF 계산

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus) #학습
tfidf_matrix = tfidf_vectorizer.transform(corpus) #transform 메서드를 데이터

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```



```

→ [[0.          0.          0.39687454 0.39687454 0.          0.79374908
      0.2344005 ]
     [0.61980538 0.          0.          0.          0.61980538 0.
      0.48133417]
     [0.4804584  0.63174505 0.          0.          0.4804584  0.
      0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}

```

- 사이킷런의 TF-IDF 메서드는 fit 메서드를 통해 학습해야 데이터의 변환 또는 추론이 가능함
- fit_transform 메서드 - 학습과 변환을 동시에 할 수도 있음
- toarray 메서드 - TF-IDF 메서드를 넘파이 배열로 변환
- vocabulary_ 속성 - TF-IDF 에 사용된 단어 사전
- 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않는다.